

ISO/IEC JTC1 SC22 WG21 P2066R9

Authors: Hans Boehm, Victor Luchangco, Jens Maurer, Michael L. Scott, Michael Spear, and Michael Wong

Reply to: Jens Maurer <Jens.Maurer@gmx.net>

Target audience: CWG, LWG

2021-09-15

P2066R9: Suggested draft TS for C++ Extensions for Minimal Transactional Memory

Introduction

This paper presents suggested wording for a future Technical Specification on a variant of Transactional Memory. See P1875R2 "Transactional Memory Lite Support in C++" for discussion.

Paper history

Changes in R1 since R0

- incorporated feedback from SG1 in Prague

Changes in R2 since R1

- turned "atomic" into a context-sensitive keyword
- added questions for TS deployment

Changes in R3 since R2

- EWG feedback: extend definition of full-expression in [intro.execution], add note about parallel execution of transactions, constant evaluations in atomic blocks are irrelevant, use "atomic do" as the introducer keywords.
- Explicitly list standard library functions required to be supported in an atomic block

Changes in R4 since R3

- Make throwing an exception from inside a transaction undefined behavior

Changes in R5 since R4

- SG5 review: Exceptions handled inside an atomic block are fine.
- SG5 review: Nearly all standard library functions may be used inside an atomic block; any other function is restricted to compiler-visible.

Changes in R6 since R5

- SG5 review: Exclude more standard library facilities causing synchronization.

Changes in R7 since R6

- EWG review: Exclude more standard library facilities causing synchronization.
- R6 was approved by EWG on 2021-04-28.

Changes in R8 since R7

- Added a section "notable design decisions" in preparation for LEWG review.

Changes in R9 since R8

- Address feedback from Hubert Tong.
- Address feedback from CWG.
- Address preliminary feedback from LWG.

Notable design decisions

A more detailed rationale for many of the decisions made here is given in [P1875](#). Here we quickly summarize design decisions leading to this proposal. These are generally motivated by the desire to produce a much simpler, more easily implementable specification. Except for the last two, these have been stable since the original SG1 discussion. All were part of the proposal during the final EWG discussion.

- This is a language, not library, proposal. Passing lambda expressions to a "run this as a transaction" function was not well-liked, and has issues with varargs access. An RAII-based library facility was considered, but also disliked by SG1, this time, among other issues, because it was unclear what this would mean if the RAII object was not stack-allocated.
- We want to leave a lot implementation-defined for the TS, to encourage implementations, and support experimentation. We do not want to again end up with a TS that is difficult to implement usefully, and thus fails to attract implementations.
- We want to focus on simple implementations that are based in hardware transactional memory, and/or use the trivial global lock implementation. We expect HTM implementations that fall back on a global lock to dominate in practice, at least initially. We want to allow more elaborate implementations based on software transactional memory but, unlike the current TS, we do not want to require such elaborate implementations.
- There is no support for explicitly declaring transaction-safety. This is a major simplification over the current Technical Specification. This does not matter for HTM or global lock implementations, which do not need to instrument code. It does make it more difficult to extract good performance from software transactional memory implementations, but there is agreement that the simplification is worth the trade-off.
- Exceptions may not escape from transactions. This dodges many complicated issues about whether transactions should commit or abort and, in the latter case, how exception objects themselves survive transaction aborts.
- The "atomic do {...}" syntax. This was the most natural syntax we (mostly EWG) could come up with that appears to be easily parseable, and does not require a new keyword. It was "atomic {...}" during SG1 discussions. "atomic do" reflects a non-unanimous consensus in EWG.
- Allow `malloc` (and hence non-escaping exceptions, and much more of the standard library) inside transactions. This was a recent change, but preceded the last EWG discussion. We previously thought that we could get by with very restricted transactions, so long as they were sufficiently general to implement N-way `compare_exchange`. Closer examination by SG5 determined that was problematic since the load operations preceding the `compare_exchange` transaction would also need to be transactions, making them expensive, at least without heroic implementation effort. Thus we concluded that it was desirable to again allow much of the standard library inside transactions. (Also discussed in more detail in P1875, but more as an open issue.)

Questions for TS deployment

- Can we strengthen some of the "implementation-defined" parts of [stmt.tx]? In which way?
- What is the facility used for in practice?
- Are there problems that might benefit from transactions, but cannot be solved with the facilities presented?
- Do implementations provide additional facilities for transactions (possibly with extra syntax) beyond those presented?
- Is it vital for the implementation that the definitions of all functions invoked within a transaction are visible?
- Are there functions in the standard library that would be useful inside a transaction, but do not work yet?

Wording changes

These wording changes are relative to the current C++ Working Draft, N4849.

In 5.10 [lex.name], add **atomic** to table 4 [tab:lex.name.special].

Change in 6.9.1 [intro.execution] paragraph 5:

A full-expression is

- ...
- an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.7.7) whose lifetime has not been extended, ~~or~~
- **the start and the end of an atomic block (8.8 [stmt.tx]), or**
- an expression that is not a subexpression of another expression and that is not otherwise part of a full-expression.

Change in 6.9.2.1 [intro.races] paragraph 6:

Atomic blocks as well as ~~Certain~~ **certain** library calls **may** *synchronize with* other **atomic blocks** **and** library calls performed by another thread.

Add a new paragraph after 6.9.2.1 [intro.races] paragraph 20:

An atomic block that is not dynamically nested within another atomic block is termed a transaction. [Note: Due to syntactic constraints, blocks cannot overlap unless one is nested within the other.] There is a global total order of execution for all transactions. If, in that total order, a transaction T1 is ordered before a transaction T2, then

- **no evaluation in T2 happens before any evaluation in T1 and**
- **if T1 and T2 perform conflicting expression evaluations, then the end of T1 synchronizes with the start of T2.**

[Note: If the evaluations in T1 and T2 do not conflict, they might be executed concurrently. -- end note]

Two actions are *potentially concurrent* if ...

Change in 6.9.2.1 [intro.races] paragraph 21:

... [Note: It can be shown that programs that correctly use mutexes, **atomic blocks**, and `memory_order::seq_cst` operations to prevent all data races and use no other synchronization operations behave as if the operations executed by their constituent threads were simply interleaved, with each value computation of an object being taken from the last side effect on that object in that interleaving. This is normally referred to as "sequential consistency". ...

Add a new paragraph after 6.9.2.1 [intro.races] paragraph 21:

[**Note: The following holds for a data-race-free program: If the start of an atomic block T is sequenced before an evaluation A, A is sequenced before the end of T, A strongly happens before some evaluation B, and B is not sequenced before the end of T, then the end of T strongly happens before B. If an evaluation C strongly happens before that evaluation A and C is not sequenced after the start of T, then C strongly happens before the start of T. These properties in turn imply that in any simple interleaved (sequentially consistent) execution, the operations of each atomic block appear to be contiguous in the interleaving. -- end note**]

Change in 6.9.2.2 [intro.progress] paragraph 1:

~~The implementation may assume that any thread will eventually do~~ **An inter-thread side effect is** one of the following:

- ~~terminate,~~
- a call to a library I/O function,
- an access through a volatile glvalue, or
- a synchronization operation or an atomic operation **([atomics])**.

The implementation may assume that any thread will eventually terminate or evaluate an inter-thread side effect. [Note: This is intended to allow compiler transformations such as removal of empty loops, even when termination cannot be proven. — end note]

Add a production to the grammar in 8.1 [expr.pre]:

```
statement:
    labeled-statement
    attribute-specifier-seqopt expression-statement
    attribute-specifier-seqopt compound-statement
    attribute-specifier-seqopt selection-statement
    attribute-specifier-seqopt iteration-statement
    attribute-specifier-seqopt jump-statement
    declaration-statement
    attribute-specifier-seqopt try-block
    atomic-statement
```

Add a new subclause before 8.8 [stmt.dcl]:

8.8 Atomic statement [stmt.tx]

atomic-statement:
atomic do *compound-statement*

An *atomic-statement* is also called an *atomic block*.

The start of the atomic block is immediately before the opening { of the *compound-statement*. The end of the atomic block is immediately after the closing } of the *compound-statement*. [**Note: Thus, variables with automatic storage duration declared in the *compound-statement* are destroyed prior to reaching the end of the atomic block; see 8.7 [stmt.jump]. -- end note**]

A goto or switch statement shall not be used to transfer control into an atomic block.

If the execution of an atomic block evaluates an inter-thread side effect (6.9.2.2 [intro.progress]) or if an atomic block is exited via an exception, the behavior is undefined.

Recommended practice: In case an atomic block is exited via an exception, the program should be terminated without invoking a terminate handler (17.9.5 [exception.terminate]) or destroying any objects with static or thread storage duration (6.9.3.4 [basic.start.term]).

If the execution of an atomic block evaluates any of the following outside of a manifestly constant-evaluated context (7.7 [expr.const]), the behavior is implementation-defined:

- an *asm-declaration* (9.10 [dcl.asm]);
- an invocation of a function **other than one of the standard library functions specified in (16.4.6.17 [atomic.use]), unless the function is inline** with a reachable definition;
- a virtual function call (7.6.1.3 [expr.call]);
- a function call, **unless overload resolution selects**
 - a **named function** (12.2.2.2.2 [over.call.func]) or
 - a **function call operator** (12.2.2.2.3 [over.call.object]), but not a surrogate call function;
- a `co_await` expression (7.6.2.3 [expr.await]), a *yield-expression* (7.6.17 [expr.yield]), or a `co_return` statement (8.7.4 [stmt.return.coroutine]);
- dynamic initialization of a block-scope variable with static storage duration; or
- dynamic initialization of a variable with thread storage duration.

[Note: The implementation **can** define that the behavior is undefined in some or all of the cases above.]

[Example:

```
unsigned int f()
{
    static unsigned int i = 0;
    atomic do {
        ++i;
        return i;
    }
}
```

Each invocation of `f` (even when called from several threads simultaneously) retrieves a unique value (ignoring **wrap-around**). -- end example]

[Note: Atomic blocks are likely to perform best where they execute quickly and touch little data. -- end note]

Add 16.4.6.17 [atomic.use]:

16.4.6.17 Functions usable in an atomic block [atomic.use]

All library functions may be used in an atomic block (8.8 [stmt.tx]), except

- **error category objects** ([syserr.errcat.objects])
- **time zone database** ([time.zone.db])
- **clocks** ([time.clock])
- **signal** ([support.signal]) **and** **raise** ([csignal.syn])
- `set_new_handler`, `set_terminate`, `get_new_handler`, `get_terminate` ([handler.functions], [alloc.errors], [exception.syn])
- **system** ([cstdlib.syn])
- `shared_ptr` ([util.smartptr.shared]) **and** `weak_ptr` ([util.smartptr.weak])
- `synchronized_pool_resource` ([mem.res.pool])
- **program-wide memory_resource objects** ([mem.res.global])

- `setjmp / longjmp` ([`csetjmp.syn`])
- `parallel algorithms` ([`algorithms.parallel`])
- `locale construction` ([`locale.cons`])
- `input/output` ([`input.output`])
- `atomic operations` ([`atomics`])
- `thread support` ([`thread`])